Operator-Input Sources and the `OperatorPose` Contract
(`mabel.sources`)

MABEL Project — Control & Teleoperation

July 11, 2026

Abstract

Every teleop device lands on one data contract. Whatever produces the operator frame — the Vision Pro ARKit skeleton over ZMQ, the iOS / web / VisionPro apps' `teleop_frame` wire format over WebSocket, a webcam skeleton from the browser studios, or a synthetic mock — `mabel.sources` converts it into a single `OperatorPose`, and everything downstream (whole-body retargeting, hand retargeting, the facade, safety) is byte-for-byte identical across devices. This short document specifies the contract, the wire dialects, and the frame conventions.

Keywords: teleoperation, wire protocol, device abstraction, ARKit.

1 Design rule

Conversion happens *once, at ingest*. No retargeter, controller, or safety layer ever branches on the device type; a new frontend is integrated by writing one converter to `OperatorPose` and nothing else (the contract is enforced culturally by this package layout and practically by the facade accepting only `OperatorPose`).

2 The `OperatorPose` contract (`mabel/types.py`)

field	type	meaning
`head`	4×4	operator head pose (ARKit world)
`right_hand` / `left_hand`	list of 4×4	21-landmark skeleton, `[0]` = wrist
`right_joints` / `left_joints`	{name: 4×4}	named ARKit skeleton (optional, richer)
`right_pinch` / `left_pinch`	m	thumb–index gap (drives clutch/grasp)
`right_elbow` / `left_elbow`	4×4	optional (enables the elbow cosine term)
`body_yaw`, `head_height`	rad, m	optional torso heading / stature signals
`timestamp`	s	source time (drives the fail-safe freshness check)

The class is duck-compatible with the historical `AVPFrame` (`.head/.right/.left/.right_joints/.left_joints`), so retargeters consume either. `signals()` exposes per-signal presence for the tracking-loss fail-safe (`mabel.safety`).

Frames. ARKit world: X right, Y up, $-Z$ forward. The fixed axis remap into the robot world (front $= -X$, left $= -Y$, up $= +Z$) is the matrix M (`config.AVP_TO_ROBOT`), applied to *relative* motion by the retargeting laws — a sign error is therefore easy to spot and flip (`whole_body_retargeting.pdf`).

3 Dialects

- `wire.py` — the apps' `teleop_frame` payload (iOS, Vision Pro, web GUI, webcam studios): head transform + per-hand anchor and/or named-joint skeletons + derived pinch; also `app_joystick` decoding the nav-stick message (left stick translate, right stick yaw + lift, deadzone 0.08). `OperatorPose.from_teleop_frame` delegates here, so *the wire dialect lives in the package, not in the transport bridge.*
- `avp.py` — the direct Vision Pro ZMQ stream (`AVPReceiver`, PUB/SUB on `tcp://127.0.0.1:5556`, raw JSON payloads; `AVPFrame` parses head/hand matrices and named joints).
- `mock.py` — `mock_frame(t)`: a synthetic operator whose wrists trace slow circles, head pans, and pinch oscillates; drives the test suite, benchmarks and data-collection tooling deterministically.

The webcam full-body path (browser studios) emits the same `teleop_frame` dialect with 21-landmark hands; the research engine converts 21-landmark to its internal payload on ingest, so webcam and headset share every downstream stage.

4 Validation

`tests/test_imports_compat.py` pins the ingest surface; `tests/test_facade_e2e.py` runs the full facade over `mock_frame` streams; `tests/test_retargeting.py` feeds synthetic calibration poses (`synthesize_pose`) through the named-joint path. Latency and rate characterisation of the live wire lives with the server (`server/tools`, teleop state-stream studies).

5 Limitations

- `mock.py` exercises geometry, not realism (rigid identity-rotation hand plates); anthropometric realism comes from `calibration.synthesize_pose`.
- The ZMQ AVP path carries no pinch/elbow extras (named joints only); those signals arrive only on the `teleop_frame` dialect.
- Time synchronisation between device and host is assumed good to the fail-safe timeout scale (0.4 s); no clock-skew estimation is done here.